Multi-Agent Orchestration Framework for Complex Process Mining Query Answering*

Gyeunggeun Doh[1,†], Sunah Min[2,†], Yujin Jung[3] and Minseok Song[1,*]

[1]Department of Industrial and Management Engineering, Pohang University of Science and Technology, Pohang 37673, South Korea

[2]Department of Social Data Science, Pohang University of Science and Technology, Pohang 37673, South Korea

[3]Department of Industrial Data Science, Pohang University of Science and Technology, Pohang 37673, South Korea

Abstract

Process mining enables systematic analysis of business processes based on event logs. However, advanced analyses often require combining multiple techniques—such as process discovery, conformance checking, and performance analysis—in multi-step workflows. Executing such workflows demands both domain expertise in process mining and technical proficiency in specialized libraries (e.g., PM4Py), creating a significant barrier to practical adoption. This paper proposes a multi-agent Large Language Model (LLM) orchestration framework that translates natural language process mining queries into executable analytical workflows. The framework decomposes a query into a sequence of process mining operations and coordinates specialized LLM agents for task planning, tool retrieval, code generation, and execution. A retrieval-augmented knowledge base provides domain-specific documentation and reusable tool code, while a self-debugging mechanism iteratively corrects runtime errors. We evaluate the framework on a newly constructed benchmark of 120 complex process mining tasks covering both single-case and object-centric event logs with varying analytical complexity. Experimental results show that the proposed multi-agent framework substantially outperforms single-agent baselines, demonstrating the effectiveness of structured agent orchestration for automated process mining query answering.

Keywords

Multi-Agent Framework, Automated Process Analysis, Code Generation

1. Introduction

Process mining provides a systematic methodology for analyzing business processes using data recorded in event logs [1]. Techniques such as process discovery, conformance checking, and performance analysis enable organizations to gain insights into operational efficiency and process behavior [2]. While individual techniques are well established, practical analytical questions often require multi-step workflows that combine several process mining operations. For example, determining whether cases that violate a mandatory control step exhibit different performance from compliant cases requires identifying violating traces, partitioning the log accordingly, and comparing performance metrics across the two groups. Such questions demand not only process mining expertise but also the ability to design and implement a sequence of interdependent analysis steps.

Executing such analyses presents substantial practical challenges. High-level business questions must be translated into well-defined analytical procedures, which in turn must be implemented through specialized libraries such as `PM4Py` [3]. Coordinating multiple analysis steps, intermediate artifacts, and library functions increases manual effort and creates a growing need for automated support that bridges the gap between natural language analytical intent and executable process mining workflows.

Recent advances in large language models (LLMs) have motivated their application to process mining. Berti et al. [4] introduced data abstractions and prompt scenarios to bridge the representation gap

ASPAI'26: Asia-Pacific Symposium on Process and AI, August 24–28, 2026, Pohang, South Korea

*Corresponding author.

†These authors contributed equally.

doh2629@postech.ac.kr (G. Doh); sunahmin@postech.ac.kr (S. Min); y8jin@postech.ac.kr (Y. Jung); mssong@postech.ac.kr (M. Song)

0009-0001-3978-2906 (G. Doh); 0000-0002-6813-8853 (M. Song)

© 2022 Copyright for this paper by its authors. Use permitted under Creative Commons License Attribution 4.0 International (CC BY 4.0).

between event logs and LLMs. Subsequent work developed evaluation strategies and benchmarks to systematically assess LLM performance on process mining tasks [5, 6], and Barbieri et al. [7] proposed a natural language interface for question-answering over event data. Despite these advances, most existing approaches rely on single-agent architectures. Such systems can handle relatively contained tasks but often struggle with multi-step analytical reasoning and lack robust mechanisms for recovering from runtime errors when interacting with specialized libraries and executable code [8]. Meanwhile, recent work on multi-agent orchestration has demonstrated strong potential for complex, tool-intensive scientific coding tasks [9], suggesting a promising direction for process mining.

To address this gap, this paper proposes a multi-agent LLM orchestration framework for complex process mining query answering. Given a natural language query and an event log, the framework generates an executable analytical workflow and returns the analysis output. The framework comprises four components: user input, a knowledge base, a multi-agent orchestration component, and an output interface. The knowledge base provides event log abstractions and a retrieval-augmented vector database [10] containing process mining documentation and reusable tools from prior successful executions. The orchestration component coordinates three specialized agents: the Supervisor constructs analysis plans and tool specifications, the Tool Generator retrieves or generates required tool code, and the Code Assembler composes and executes the final analysis script. A self-debugging loop enables the framework to revise plans and tools when runtime errors occur.

We evaluate the framework on a newly constructed benchmark of 120 complex process mining tasks spanning single-case and object-centric event logs with queries ranging from single-intent to five-intent analytical complexity. Experimental results show that the proposed framework substantially outperforms single-agent baselines on both event log types, providing clear evidence that multi-agent orchestration is more effective than single-agent generation for executable process mining query answering.

The contributions of this paper are threefold:

1. A multi-agent LLM orchestration framework that translates natural language process mining queries into executable analytical workflows through coordinated task planning, tool retrieval, code generation, and self-debugging.
2. A retrieval-augmented knowledge base that provides domain-specific documentation, supports the generation of reusable tools, and stores validated tools for future reuse.
3. A benchmark of 120 complex process mining tasks covering both single-case and object-centric event logs, together with an empirical evaluation demonstrating the effectiveness of the proposed framework.

The remainder of this paper is organized as follows. Section 2 presents the proposed framework. Section 3 describes the benchmark and experimental evaluation. Section 4 concludes the paper and discusses directions for future work.

2. Multi-Agent Orchestration Framework

This section presents the proposed multi-agent orchestration framework for complex process mining query answering. Figure 1 provides an overview of the end-to-end flow. Given a natural language process mining query and an event log, the framework produces analysis output by executing an automatically generated analytical workflow. The framework comprises four components: (i) user input, (ii) a knowledge base, (iii) a multi-agent orchestration component, and (iv) an output interface. The knowledge base provides event log abstractions and a RAG vector database offering retrieval-augmented access to the PM4Py manual and reusable tools from prior executions. The orchestration component coordinates specialized LLM agents that plan the workflow, define required tools, assemble executable code, and manage execution feedback in a controlled loop. The following subsections describe each component in detail.

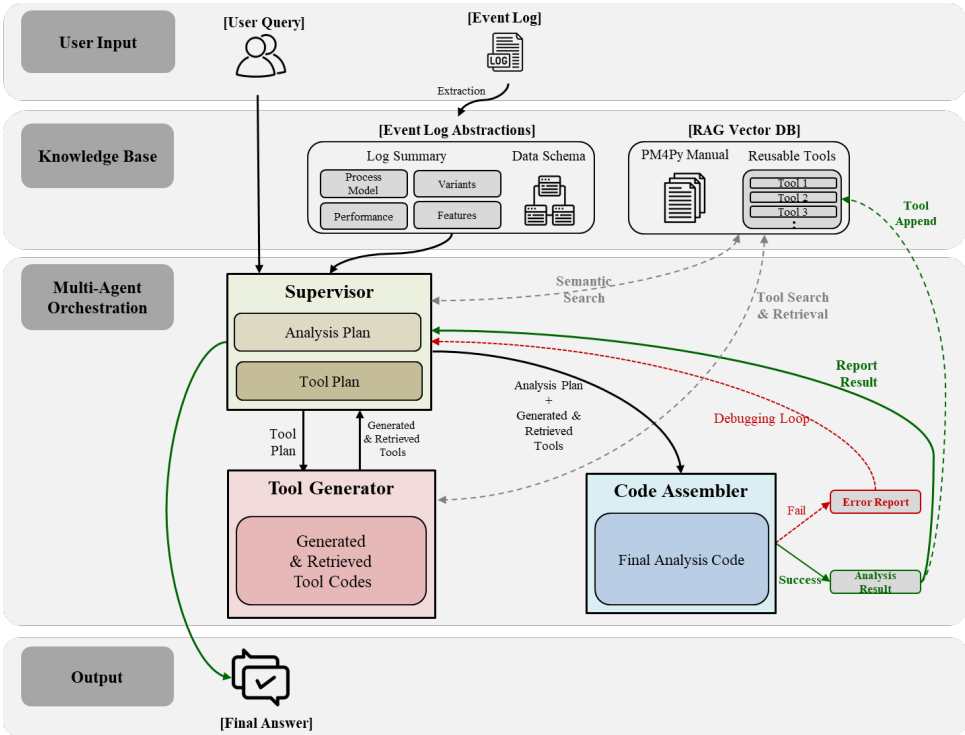

Figure 1: Overview of the proposed multi-agent orchestration framework for complex process mining query answering.

2.1. User Input

User input consists of a natural language process mining query and an event log, which may be either a single-case event log or an object-centric event log (OCEL). The framework uses the event log as input without modifying the original data.

Importantly, the full event log is not provided to the LLM agents, as placing large logs directly in the context window is both inefficient and costly. Instead, the framework derives compact representations of the event log and provides these to the orchestration component. The Code Assembler loads the full event log locally when executing generated analysis code.

2.2. Knowledge Base

The knowledge base provides two types of information required for query answering. The first is the event log abstractions, which capture the structure and basic behavior of the input data. The second is a retrieval-augmented vector database that provides access to process mining documentation and a repository of reusable tools.

2.2.1. Event Log Abstractions

Event log abstractions summarize the input event log to provide compact context for planning and code generation. As shown in Figure 1, the abstractions consist of two complementary parts.

The *log summary* captures commonly used process mining views derived from the event log, including a coarse process model view, performance summaries, representative variants, and feature statistics. These summaries enable the orchestration component to select appropriate analysis steps and parameters without requiring direct access to the raw event log.

The *data schema summary* captures the tables, attributes, and relationships needed for implementation. For a single-case event log, it identifies the table structure and relevant attributes for case, activity, and timestamp handling. For an OCEL, it captures the structure of event, object, and relation tables along

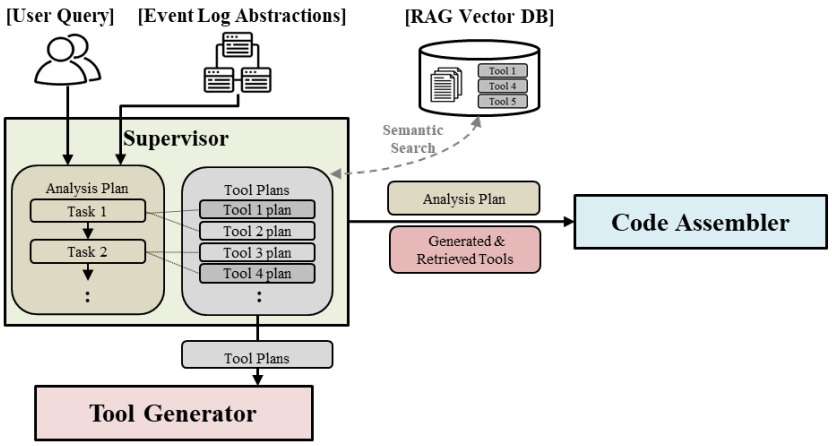

Figure 2: The Supervisor agent translating a process mining query into an analysis plan and tool plans.

with available object types. By providing the exact attribute names and data structures that generated code must reference, the schema summary reduces errors during code generation.

2.2.2. RAG Vector DB

The framework maintains a retrieval-augmented vector database that stores two categories of content.

The first category is *process mining documentation*, including the PM4Py manual and supplementary documentation describing analysis concepts and library usage patterns. Rather than placing full documentation in the prompt, the framework retrieves relevant fragments on demand to support planning and code generation—for instance, identifying relevant algorithms, function signatures, and parameter conventions.

The second category is a *repository of reusable tools*, consisting of tool code from prior successful executions stored with metadata for semantic retrieval. When a new query requires similar functionality, the framework can retrieve and reuse existing tool implementations instead of generating them from scratch.

2.3. Multi-Agent Orchestration

The orchestration component converts a process mining query into an executable analytical workflow by coordinating three specialized LLM agents: Supervisor, Tool Generator, and Code Assembler. Each agent is assigned a distinct role through a dedicated prompt that specifies its responsibilities and the artifacts it must produce. This design separates planning, tool construction, and code assembly into modular steps with structured hand-offs between agents.

2.3.1. Supervisor

The Supervisor acts as the central planner and coordinator. As illustrated in Figure 2, the Supervisor receives the process mining query together with the knowledge base outputs—event log abstractions and documentation fragments retrieved via semantic search over the RAG vector database—and produces two planning artifacts: an *analysis plan* and *tool plans*.

The analysis plan specifies the analytical workflow at a high level by decomposing the query into a sequence of process mining operations. Each operation is represented as a workflow step mapped to a corresponding analysis task. The plan states intermediate artifacts, step dependencies, and execution order in a pseudo-code style, referencing data views and conditions from the data schema summary and event log abstractions. As shown in Figure 2, each analysis task serves as the basis for defining the tools required to execute it.

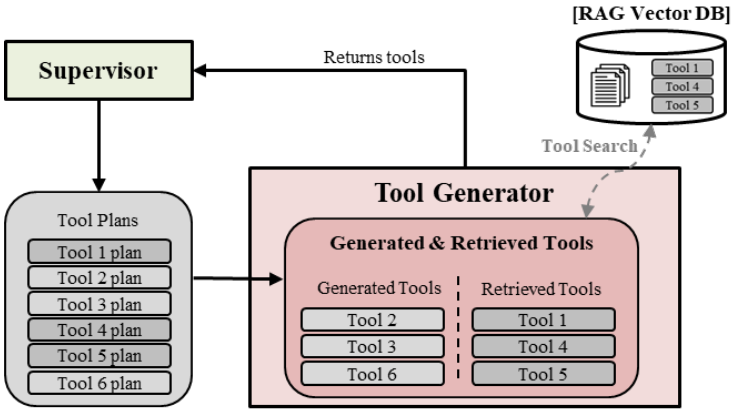

Figure 3: The Tool Generator agent resolving tool plans through retrieval or generation.

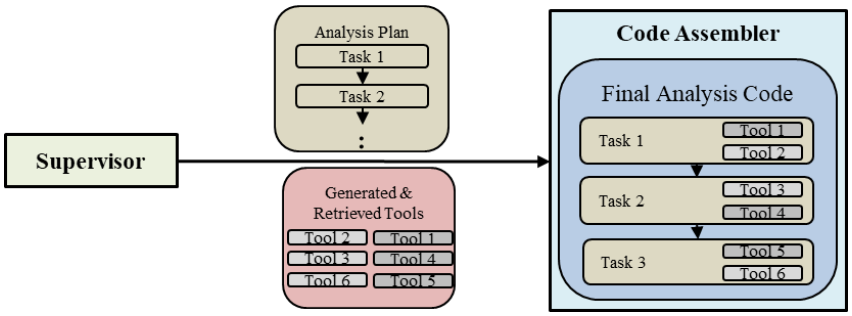

Figure 4: The Code Assembler agent composing the final executable script from the analysis plan and tools.

Tool plans specify the tools needed to implement each analysis task. Each tool specification describes the tool purpose, inputs, outputs, parameters, and lightweight validation requirements (e.g., expected columns, type compatibility, non-empty results) to reduce runtime errors. The Supervisor sends the tool plans to the Tool Generator for retrieval or generation, receives the resulting tool code, and forwards both the analysis plan and tool code to the Code Assembler.

2.3.2. Tool Generator

The Tool Generator resolves each tool plan into executable tool code, as illustrated in Figure 3. For each tool plan, the agent first searches the RAG vector database for an existing tool that matches the required functionality. If a suitable tool is found, it is reused directly; otherwise, the Tool Generator retrieves relevant documentation fragments and generates new tool code. The resulting set of reused and newly generated tools is returned to the Supervisor for downstream code assembly.

2.3.3. Code Assembler

The Code Assembler receives the analysis plan and tool code, and composes them into a single executable script. As illustrated in Figure 4, the agent maps the ordered tasks in the analysis plan to concrete tool invocations, materializes intermediate artifacts required by subsequent tasks, and enforces the specified execution order. The resulting analysis code is then ready for local execution with the event log as input.

2.4. Code Execution and Output

After code assembly, the framework executes the generated script locally with the event log as input. Execution produces either analysis outputs or an error report containing runtime error information.

Table 1
Benchmark Dataset Composition by Event Data Type and Intent Count.

Event Data Type	Total	Intent Count				
		1	2	3	4	5
Single-case event log	60	20	10	10	10	10
Object-centric event log (OCEL)	60	20	10	10	10	10
Total	120	40	20	20	20	20

2.4.1. Execution Feedback and Tool Reuse

The framework uses execution feedback for both failure recovery and future reuse. When execution fails, the Supervisor receives the error report and revises the analysis plan, tool plans, or both—potentially adjusting step ordering, parameter choices, data views, or tool specifications. The framework then repeats tool generation and code assembly under a predefined retry budget, terminating when execution succeeds or the retry limit is reached.

Upon successful execution, the framework stores the validated tools in the RAG vector database. Tools are stored only after successful execution to prevent caching faulty implementations. Future queries requiring similar functionality can then retrieve stored tools without regenerating them.

2.4.2. Outputs

The framework produces two outputs: (1) the final answer, which reports the analysis result and includes optional artifacts such as discovered models, visualizations, or summary tables; and (2) the executed analysis code.

3. Evaluation

This section describes the benchmark dataset, experimental setup, and results. The evaluation covers both traditional single-case event logs and object-centric event logs with varying analytical complexity, and compares the proposed multi-agent framework against single-agent baselines under controlled input settings. A query is counted as solved only when the generated workflow executes successfully and produces the expected answer for the corresponding query–answer pair. The framework implementation and benchmark dataset are publicly available.[1]

3.1. Benchmark Dataset

The benchmark consists of 120 query–answer pairs: 60 over traditional single-case event logs and 60 over object-centric event logs (OCEL). Each instance contains a natural language process mining query and a reference answer. For the single-case setting, we use the `BPI Challenge 2017` [2] event log. For the object-centric setting, we use the `order-management` [3] OCEL dataset.

We define an *intent* as an analysis objective implementable by one or more process mining operations, and use intent count to characterize the analytical complexity of each query. The benchmark defines five intents: *process discovery* (deriving explicit representations of process behavior), *variant analysis* (characterizing and comparing execution patterns), *performance analysis* (quantifying time-related aspects), *conformance checking* (assessing compliance with reference behavior or rules), and *resource-oriented analysis* (examining resource participation and its relation to behavior and performance). In the OCEL subset, resource-oriented analysis is replaced by *object-oriented analysis*, which examines object types, event–object relations, and object-centric views of behavior and performance.

[1] https://github.com/
[2] https://doi.org/10.4121/uuid:5f3067df-f10b-45da-b98b-86ae4c7a310b
[3] https://doi.org/10.5281/zenodo.18373906

Table 2
Solved query counts on the benchmark dataset and input settings of each method.

Framework	Input Setting	Single-case / 60	OCEL / 60
Single-agent Baseline 1	Query + data schema summary	10 (16.7%)	5 (8.3%)
Single-agent Baseline 2	Query + data schema summary + log summary	9 (15.0%)	2 (3.3%)
Single-agent Baseline 3	Query + data schema summary + log summary + RAG vector DB	7 (11.7%)	4 (6.7%)
Multi-agent Orchestration (Ours)	Query + data schema summary + log summary + RAG vector DB	27 (45.0%)	11 (18.3%)

A single-intent query targets one analytical objective and typically requires a single primary operation with limited supporting steps. A multi-intent query combines several intents, requiring a chained workflow in which outputs of earlier steps feed into later ones. For example, a three-intent query might first select cases by a performance measure, then identify the dominant variant among those cases, and finally rank resources observed in that subset. Table 1 summarizes the benchmark composition by event data type and intent count.

To illustrate, a single-intent query such as "Identify the slowest 10% of cases by throughput time" targets performance analysis alone. A multi-intent query such as "Identify the slowest 10% of cases by throughput time, determine the dominant variant among those cases, and report the top resources observed in that subset" combines performance analysis, variant analysis, and resource-oriented analysis. An OCEL multi-intent query such as "Select an object type, identify objects that violate a specified rule over related events, and compare cycle-time statistics between violating and non-violating objects" combines object-oriented analysis, conformance checking, and performance analysis.

By covering both single-case and object-centric event logs with queries ranging from single-intent to five-intent settings, the benchmark provides a controlled testbed for assessing both basic process mining capability and the ability to handle chained, multi-step analytical reasoning.

3.2. Evaluation Results

For a fair comparison, all methods use the same underlying LLM (GPT-4.5 Codex) and are evaluated on the same benchmark instances under the same execution setting. The methods differ only in the input context provided and in whether reasoning is handled by a single agent or the proposed multi-agent framework.

The three single-agent baselines isolate the effect of input context under a single-agent architecture. Baseline 1 receives only the query and data schema summary, testing whether minimal structural information suffices. Baseline 2 additionally receives the log summary, providing compact behavioral and statistical abstractions. Baseline 3 further adds retrieval access to the RAG vector database, representing the strongest single-agent setting with both event-log abstractions and external documentation. The proposed framework receives the same input as Baseline 3 but distributes the task across specialized agents and additionally incorporates reusable tools.

Table 2 reports the results. The proposed framework solves 27 out of 60 single-case queries (45.0%) and 11 out of 60 OCEL queries (18.3%). The best-performing single-agent baseline solves only 10 (16.7%) and 5 (8.3%), respectively. The multi-agent framework thus achieves a 2.7× improvement on single-case queries and a 2.2× improvement on OCEL queries relative to the strongest baseline.

A notable observation from the single-agent baselines is that richer context does not consistently improve performance. Baselines 2 and 3 receive more information than Baseline 1 yet achieve comparable or lower solve rates. This pattern is consistent with the *context distraction* phenomenon observed in prior work [11]: additional context may increase the reasoning burden on a single agent, causing it to focus on peripheral information rather than identifying the correct sequence of process mining operations.

The proposed multi-agent framework mitigates this difficulty by distributing responsibilities across specialized agents. Rather than requiring a single agent to simultaneously interpret the query, plan the workflow, select tools, and compose executable code, the framework allows each agent to operate on a focused objective: the Supervisor handles planning, the Tool Generator resolves tools, and the Code Assembler composes the final script. This modular decomposition reduces the reasoning burden on any individual agent and supports more coherent multi-step reasoning.

The results also reveal that the OCEL subset is substantially more challenging for all methods. This gap reflects the additional complexity of object-centric process mining—object-type selection, traversal of event–object relations, construction of object-centric views, and reasoning about interactions among multiple object types rather than a single case notion.

Finally, the absolute solve rates indicate that the benchmark remains challenging even for the proposed framework, with many queries unsolved in both settings. This suggests that complex process mining query answering is far from a solved problem, with further improvements needed in object-centric tool coverage, long-horizon workflow planning, and robust cross-step coordination. Nevertheless, the results provide clear evidence that multi-agent orchestration substantially outperforms single-agent generation for executable process mining query answering.

4. Conclusion

This paper proposed a multi-agent LLM orchestration framework that translates natural language process mining queries into executable analytical workflows. By combining event log abstractions, a retrieval-augmented knowledge base, and three specialized agents for workflow planning, tool generation, and code assembly, the framework automates the end-to-end process from query to analysis output.

Evaluation on a benchmark of 120 complex queries—spanning single-case and object-centric event logs with up to five analytical intents—showed that the proposed framework substantially outperforms single-agent baselines. Notably, the results also demonstrated that enriching the input context of a single agent does not consistently improve performance, whereas distributing responsibilities across specialized agents yields clear gains. These findings suggest that structured multi-agent orchestration is a more effective paradigm than single-agent generation for complex process mining tasks requiring multi-step reasoning.

The benchmark remains challenging, particularly for OCEL queries, indicating that complex process mining query answering is far from a solved problem. Future work will focus on expanding object-centric tool coverage, improving long-horizon workflow planning, strengthening tool retrieval and reuse, and exploring fine-tuning strategies for process-mining-specific planning and code generation.

Acknowledgments

This work was supported by the National Research Foundation of Korea (NRF) grant funded by the Korea government (MSIT) (No. RS-2024-00357330).

Declaration on Generative AI

During the preparation of this work, the authors used GPT-5 and Gemini-3 for grammar and spelling checks. After using these tools, the authors reviewed and edited the content as needed and take full responsibility for the publication's content.

References

[1] W. Van Der Aalst, Process mining: Overview and opportunities, ACM Transactions on Management Information Systems (TMIS) 3 (2012) 1–17.

[2] W. M. Van der Aalst, Business process management: a comprehensive survey, International Scholarly Research Notices 2013 (2013) 507984.

[3] A. Berti, S. van Zelst, D. Schuster, Pm4py: A process mining library for python, Software Impacts 17 (2023) 100556.

[4] A. Berti, D. Schuster, W. M. van der Aalst, Abstractions, scenarios, and prompt definitions for process mining with llms: A case study, in: International conference on business process management, Springer, 2023, pp. 427–439.

[5] A. Berti, H. Kourani, H. Häfke, C.-Y. Li, D. Schuster, Evaluating large language models in process mining: Capabilities, benchmarks, and evaluation strategies, in: International conference on business process modeling, development and support, Springer, 2024, pp. 13–21.

[6] A. Berti, H. Kourani, W. M. van der Aalst, Pm-llm-benchmark: Evaluating large language models on process mining tasks, in: International Conference on Process Mining, Springer, 2024, pp. 610–623.

[7] L. Barbieri, K. Stroeh, E. R. Madeira, W. M. van der Aalst, An llm-based q&a natural language interface to process mining, in: International Conference on Process Mining, Springer, 2024, pp. 5–17.

[8] C. Sun, S. Huang, D. Pompili, Llm-based multi-agent decision-making: Challenges and future directions, IEEE Robotics and Automation Letters (2025).

[9] S. Raghavan, T. Mallick, Mosaic: Multi-agent orchestration for task-intelligent scientific coding, arXiv preprint arXiv:2510.08804 (2025).

[10] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, et al., Retrieval-augmented generation for knowledge-intensive nlp tasks, Advances in neural information processing systems 33 (2020) 9459–9474.

[11] N. F. Liu, K. Lin, J. Hewitt, A. Paranjape, M. Bevilacqua, F. Petroni, P. Liang, Lost in the middle: How language models use long contexts, Transactions of the association for computational linguistics 12 (2024) 157–173.